On Efficient Computations of $y^2 = x^3 + b/\mathbb{F}_p$ for Primes $p \equiv 1 \pmod 3$

Guangwu Xu[*], Wei Yu[†], Ke Han[‡], Pengfei Lu[§]

Abstract

Since its introduction, Solinas' window τ-NAF algorithm has been a landmark method for accelerating scalar multiplication on Koblitz curves over binary fields. A long-standing open problem has been to identify a suitable family of elliptic curves over prime fields for which the window τ-NAF approach can be effectively extended. In this paper, we settle this problem by establishing such an extension for the family $E_b : y^2 = x^3 + b$ over $\mathbb{F}_p$ with prime $p \equiv 1 \pmod 3$. This family includes several practically important curves used in blockchain applications (e.g., secp256k1) and pairing-based cryptography (e.g., BN254 and BLS12-381). By considering a nonzero nonunit element of minimal norm in the ring of Eisenstein integers $\mathbb{Z}[\omega]$, we identify the endomorphism $\tau = 1 - \omega$ as a natural choice for τ-adic scalar multiplication on $E_b/\mathbb{F}_p$. In Jacobian projective coordinates, the map τP can be evaluated using only $6\mathbf{M}$ (where $\mathbf{M}$ denotes a field multiplication). This also yields a new point-tripling formula requiring only $10\mathbf{M}$, improving upon the previous best cost of $15\mathbf{M}$. Furthermore, we optimize the pre-computation stage by choosing a set of coefficients invariant under the unit group $U \subset \mathbb{Z}[\omega]$. Exploiting this sixfold symmetry reduces the pre-computation cost by approximately five-sixths. The U-invariant structure also plays an important role in further accelerating window τ-NAF evaluation. Our optimized method achieves performance improvements of 16.7%, 17.6%, and 18% over the current state-of-the-art GLV method for 256-, 384-, and 512-bit group orders, respectively. We also develop a regular window τ-NAF variant as a countermeasure against side-channel attacks. Compared with the regularized GLV method, this variant reduces the scalar multiplication cost by 17.7%, 19.8%, and 20.9% for 256-, 384-, and 512-bit group orders, respectively.

[*]SCST, Shandong University, China, e-mail: gxu4sdq@sdu.edu.cn(Corresponding author)
[†]IIE, Chinese Academy of Sciences, China, e-mail: yuwei@iie.ac.cn(Corresponding author)
[‡]SCST, Shandong University, China, e-mail: 202237084@mail.sdu.edu.cn
[§]SCST, Shandong University, China, e-mail: PengfeiLu@mail.sdu.edu.cn

1 Introduction

Since its introduction by Solinas in the late 1990s, the window τ-adic non-adjacent form (τ-NAF) algorithm [32] has been a landmark method for accelerating scalar multiplication on Koblitz curves [21] over binary fields. By exploiting the Frobenius endomorphism τ, Solinas showed that scalar multiplication can be performed without point doublings, relying instead on the cheaper τ map and a sparse set of point additions. The method exploits the complex multiplication structure of Koblitz curves by treating τ as an algebraic integer in the quadratic imaginary field associated with $\mathbb{Z}[\tau]$. The use of efficiently computable endomorphisms was subsequently developed by Gallant, Lambert, and Vanstone into the influential GLV method [16], which provides fast scalar multiplication on a broad class of elliptic curves, including curves over prime fields.

A long-standing open problem has been whether Solinas' window τ-NAF framework admits an effective counterpart over prime fields. As the original window τ-NAF itself applies to the distinguished family of binary Koblitz curves, the corresponding problem over prime fields is not to construct such a method for arbitrary elliptic curves, but rather to identify a suitable family of prime-field curves together with an efficiently computable endomorphism that can serve as a τ-adic radix.

For elliptic curves over prime fields, the Frobenius map is the identity and hence provides no computational benefit. The central question is therefore whether there exists a suitable, efficiently computable (complex) endomorphism τ that can replace the role played by Frobenius on binary Koblitz curves and support an efficient window τ-NAF method. According to the discussion in [16], such a τ should satisfy the following requirements: (1) evaluating τ on a point is no more expensive than a point doubling; (2) the τ-adic expansion of the scalar is not significantly longer than its binary expansion; and (3) $N(\tau) > 1$, where $N(\tau) = \tau\overline{\tau}$ is its norm.

The family of curves

$$E_b : y^2 = x^3 + b$$

over $\mathbb{F}_p$ where the prime $p \equiv 1 \pmod 3$ is of great practical significance. The Standards for Efficient Cryptography [38] include three curves of this form: secp192k1, secp224k1, and secp256k1. In particular,

$$\text{secp256k1} : y^2 = x^3 + 7 \quad \text{over } \mathbb{F}_p,$$

where $p = 2^{256} - 2^{32} - 977$, is a 256-bit prime-field curve and is included in the NIST recommendations for discrete-logarithm-based cryptography [37]. Several pairing-friendly curves, including BN254 and BLS12-381, also belong to this family [3, 10]. These curves have been widely deployed in blockchain and pairing-based applications. Since their j-invariant is 0, they possess complex multiplication associated with the ring of Eisenstein integers $\mathbb{Z}[\omega]$, where ω is a primitive cube root of unity.

In this paper, we settle the above open problem by constructing a window τ-NAF framework for the family $E_b/\mathbb{F}_p$. The key is to consider a nonzero nonunit element of minimal norm in $\mathbb{Z}[\omega]$, which naturally leads to

$$\tau = 1 - \omega.$$

This endomorphism satisfies the three requirements above and can be evaluated using only $6\mathbf{M}$ in Jacobian projective coordinates, making it cheaper than a point doubling. The resulting τ-adic structure is further strengthened by efficient augmented-τ and point-tripling operations and by a precomputation strategy exploiting the sixfold symmetry of the unit group of $\mathbb{Z}[\omega]$. Thus, the construction is not merely a theoretical prime-field analogue of Solinas' window τ-NAF: its optimized implementation also outperforms the state-of-the-art GLV method.

1.1 Related Work

In 1991 Koblitz proposes the following CM-curves

$$E_a : y^2 + xy = x^3 + ax^2 + 1 \text{ over } \mathbb{F}_{2^m}, \quad a \in \mathbb{F}_2$$

for cryptography because they are computationally efficient. They are a classic example of using an easy-to-compute endomorphism (the Frobenius map $\tau_2(x, y) = (x^2, y^2)$) in scalar multiplication. This map satisfies $\tau_2^2 - (-1)^{1-a}\tau_2 + 2 = 0$, and $\mathbb{Z}[\tau_2]$ is the endomorphism ring of E_a.

Solinas later extended Koblitz's idea into the celebrated window τ_2-NAF method, which uses reduction to convert an integer k into $k_1 + k_2\tau_2$ with $|k_1|, |k_2| \lessapprox 2^{\frac{m}{2}}$ and $kP = (k_1 + k_2\tau_2)P$. The method then produces a (width w) sparse expansion:

$$k_1 + k_2\tau_2 = \sum_{i=0}^{l-1} \varepsilon_i u_i \tau_2^i, \quad \varepsilon_i \in \{-1, 1\},$$

where each u_i is from a chosen coefficient set C and at most one u_i in any block of length w is nonzero. This yields

$$kP = \sum_{i=0}^{l-1} \epsilon_i \tau_2^i(u_i P).$$

using pre-computed values cP for $c \in C$.

Choosing such coefficient sets is more involved than in the integer case. Blake, Murty, and Xu developed a general theory for τ_2-adic representations in $\mathbb{Q}(\tau_2)$ [6] and

extended it to all Euclidean imaginary quadratic fields [7, 8], allowing flexible and efficient pre-computation for Koblitz curves [34, 36].

Gallant, Lambert, and Vanstone [16] introduced a fast scalar multiplication method (the GLV method) for elliptic curves equipped with a efficiently computable endomorphism Φ. Let P be a point of prime order n and let $\lambda < n$ be the integer such that $\Phi(P) = \lambda P$. Then any integer $k \in [1, n-1]$ can be decomposed as $k \equiv k_1 + k_2\lambda \pmod{n}$ with $|k_1|, |k_2| = O(\sqrt{n})$, yielding

$$kP = k_1 P + k_2 \Phi(P).$$

The right-hand side is evaluated using simultaneous multiple scalar multiplication or interleaving [17, §3.5], which reduces the number of point additions by nearly half compared to a straightforward scalar multiplication. Currently, for the family of curves E_b over prime field $\mathbb{F}_p$ with $p \equiv 1 \pmod 3$, the GLV method is the-state-of-the-art for scalar multiplication.

It is useful to distinguish the roles played by endomorphisms in the two approaches. A scalar decomposition step already appears in Solinas' construction, where it is referred to as *reduction*: an integer scalar k is first reduced to an element $k_1 + k_2\tau_2$ before the resulting element is further expanded in window τ_2-NAF. The GLV method exploits this decomposition principle over a much broader class of curves, using an efficiently computable endomorphism to obtain a short multidimensional scalar decomposition. Solinas' window τ_2-NAF, however, goes further by using the endomorphism itself as the radix of the subsequent scalar expansion. It is this stronger τ-adic structure, rather than scalar decomposition alone, whose effective realization over prime fields is addressed in the present work.

Galbraith, Lin, and Scott [15] proposed a different approach (the GLS method) using quadratic twists over $\mathbb{F}_{p^2}$. Their construction yields two independent endomorphisms (the Frobenius map and a twist map) that can be simultaneously exploited. Subsequently, the GLV and GLS techniques were combined to obtain a four-dimensional decomposition on certain curves:

$$kP = k_1 P + k_2 \Phi(P) + k_3 \Psi(P) + k_4 \Psi\Phi(P),$$

with $|k_i| = O(n^{1/4})$.

Further developments of GLV/GLS methods include the work of Bos, Costello, Hisil, and Lauter [9], Hu, Longa, and Xu [19], Longa and Sica [25, 26], and Smith [30].

A generalization of the binary window non-adjacent form (window NAF) to window ℓ-NAF for any integer $\ell \geq 2$ was proposed in [33]. A more efficient way to compute the multiplication-by-ℓ map using isogeny decomposition was studied in [14] and applied to window ℓ-NAF to speed up scalar multiplication. We note that window ℓ-NAF and our window τ-NAF are different: ℓ is a rational integer, whereas τ

is an algebraic integer in a Euclidean imaginary quadratic field. In general, when applicable, the GLV method outperforms ℓ-NAF. For a 256-bit subgroup of $E_b(\mathbb{F}_p)$, even using our efficient tripling formula in window 3-NAF would cost over 2400**M**, while GLV costs under 1800**M**. Since our window τ-NAF improves upon GLV, it is therefore better than ℓ-NAF for $E_b/\mathbb{F}_p$.

Recently, the Integer Sub-Decomposition (ISD) method is proposed in [2] to handle cases where $|k_1|$ or $|k_2| > \sqrt{n}$ in the GLV decomposition $k \equiv k_1 + k_2\lambda \pmod{n}$. The paper tests ISD on secp192k1 against the window NAF (width 3) and reports a 15% speedup. We remark that secp192k1 is an instance of E_b and GLV is the state-of-the-art, but the paper fails to compare its results to a standard GLV implementation. On secp192k1, GLV is much more than 15% faster than the window NAF, and hence also faster than ISD. Moreover, it was shown in [3, 19] that for curves like secp192k1, $|k_1|, |k_2| \leq \frac{2}{3}\sqrt{n}$ holds, making ISD unnecessary.

1.2 Our Contribution

Our contributions are summarized as follows:

1. **Identification of τ:** We consider non-zero and non-unit elements of $\mathbb{Z}[\omega]$ of minimal norm, where ω is a primitive cube root of unity. This yields the candidate $\tau = 1-\omega$ for a window τ-NAF. A simple affine formula for evaluating $\tau(P)$ was given by Antony and Kamarulhaili in [1]. We compute $\tau(P)$ in Jacobian projective coordinates by using only $3\mathbf{S} + 3\mathbf{M}$ (where **S** denotes a field squaring and **M** a field multiplication), which is cheaper than a point doubling. Based on results from [3, 7], we obtain an efficient τ-adic expansion of the scalar. Since $N(\tau) = 3 > 1$, the endomorphism τ satisfies the three conditions in [16] and is suitable for our purpose.

2. **More efficient formulas:** Using the efficient Jacobian projective representation of $\tau(P)$, we design a tripling ($3P$) routine costing $4\mathbf{S} + 6\mathbf{M}$. According to the Explicit-Formulas Database [5] (for short Weierstrass curves with $a_4 = 0$ in Jacobian coordinates), the previous best operation counts for tripling on $E_b/\mathbb{F}_p$ are $10\mathbf{S} + 5\mathbf{M}$ or $7\mathbf{S} + 8\mathbf{M}$.

   Our $3P$ formula can be used in an optimized version of our window τ-NAF. It is also of independent interest because $3P$ is a main ingredient in double-base chain methods for scalar multiplication, where greater efficiency can be achieved using the new formula. We note that tripling on Jacobi-intersection curves (type 1 and 2) is very efficient ($10\mathbf{S} + 4\mathbf{M}$ and $12\mathbf{S} + 3\mathbf{M}$, respectively [5, 18]), but those curves are different from E_b.

In Jacobian projective coordinates, given ωP, we can evaluate $\tau(P)$ using $2\mathbf{S} + 3\mathbf{M}$; we call this an *augmented-τ* operation. This creates further savings for our window τ-NAF.

We also obtain efficient expressions for the endomorphisms $\rho = 2 - \omega$ and $3 - \omega$. In Jacobian projective coordinates, computing ρP costs $6\mathbf{S} + 7\mathbf{M}$, which is less expensive than a point addition. ρP is used in the pre-computation phase of window τ-NAF. Since $N(2 - \omega) = 7$, we can compute $7P$ using $12\mathbf{S} + 14\mathbf{M}$.

3. **U-invariant symmetry:** We develop a pre-computation strategy that is invariant under the unit group $U \subset \mathbb{Z}[\omega]$. By exploiting this sixth-order symmetry, we reduce the number of coefficients required in the pre-computation phase by about five-sixths, significantly lowering memory and computational overhead. The U-invariant property also plays an important role in using the tripling and augmented-τ operations to optimize the window τ-NAF and regular window τ-NAF.

4. **Performance benchmarks:** We demonstrate that our optimized window τ-NAF algorithm outperforms the standard GLV method by 16.7%, 17.6%, and 18% for group orders of 256, 384, and 512 bits, respectively. This is based on the precise number of $\mathbf{M}$ used for the three sizes, for both of our optimized window τ-NAF method and the GLV method, as shown in Table 1:

Table 1: Costs of Field Multiplication for Window τ-NAF vs. GLV

Bit size of the group order	Our method	GLV method
256	1476$\mathbf{M}$	1774$\mathbf{M}$
384	2168$\mathbf{M}$	2631$\mathbf{M}$
512	2859$\mathbf{M}$	3489$\mathbf{M}$

5. **Side-channel resistance:** We provide a regular variant of the window τ-NAF algorithm designed for constant-time execution and uniform power traces, mitigating risks from simple power analysis. This regularized version significantly outperforms the regular GLV method, reducing costs by 17.7%, 19.8%, and 20.9% for 256-bit, 384-bit, and 512-bit group orders, respectively. Table 2 contains the numbers of $\mathbf{M}$ used for the three sizes, for both of our regular window τ-NAF method and the regular GLV method:

The rest of this paper is organized into four sections. Section 2 provides preliminaries and develops some necessary tools. The derivation of efficient formulas is

Table 2: Costs of Field Multiplication for Regular Methods

Bit size of the group order	Our regular method	regular GLV method
256	1683**M**	2047**M**
384	2410**M**	3007**M**
512	3137**M**	3967**M**

described in Section 3. The main algorithms and related discussions are presented in Section 4. We conclude the paper in Section 5.

2 Preliminaries

Let $\omega = \frac{-1+\sqrt{-3}}{2}$ be the primitive complex cube root of unity, satisfying $\omega^2+\omega+1 = 0$. The curves we consider are of the form

$$E_b : y^2 = x^3 + b \quad \text{over } \mathbb{F}_p,$$

where p is a prime with $p \equiv 1 \pmod 3$ and $b \in \mathbb{F}_p^*$. For such a prime, there exists a primitive cubic root of unity β in $\mathbb{F}_p$, i.e., $\beta \neq 1$ and $\beta^3 \equiv 1 \pmod p$.

According to a refinement of Rajwade's point counting formula [29] reported in [35], one can efficiently compute a prime $\pi := c' + d'\omega \in \mathbb{Z}[\omega]$ such that $p = N(\pi)$ and $\#E_b(\mathbb{F}_p) = N(\pi - 1)$.

Suppose $\#E_b(\mathbb{F}_p) = hn$ for some prime n and a small cofactor h. In practical applications, it is often required that $n \equiv 1 \pmod 3$. In this case, we can find integers c, d such that $N(c + d\omega) = n$, namely

$$n = c^2 - cd + d^2. \tag{1}$$

We shall assume $n \equiv 1 \pmod 3$ with $n = c^2 - cd + d^2$ throughout the paper. Let $G \subset E_b(\mathbb{F}_p)$ be a subgroup of order n.

For $P = (x, y) \in E_b(\mathbb{F}_p) \setminus \{\mathcal{O}\}$, the endomorphism Φ is given by $\Phi(P) = (\beta x, y)$. In terms of the complex multiplication, we write $\Phi = \omega$. Since G is cyclic of prime order n, Φ acts on G as scalar multiplication by some integer $\lambda < n$; i.e., $\Phi(P) = \lambda P$ for all $P \in G$. Hence for any $(x, y) \in G \setminus \{\mathcal{O}\}$

$$\lambda(x, y) = (\beta x, y).$$

It follows that λ is a primitive cube root of unity modulo n, and $\lambda^2(x, y) = (\beta^2 x, y)$.

Scalar reduction. The goal of scalar reduction (or *decomposition*) is to compute, for a given scalar $k < n$, the representation

$$k \equiv k_1 + k_2\lambda \pmod{n} \tag{2}$$

with integers k_1, k_2 both of size about $\sqrt{n}$. We adopt the method from [11] (see also [19]). The procedure starts by computing

$$x = \frac{k(c-d)}{n}, \quad y = \frac{-kd}{n},$$

and then setting

$$q_1 = \left\lfloor \frac{\lfloor x+y \rfloor + \lfloor 2x-y \rfloor + 2}{3} \right\rfloor, \qquad q_2 = \left\lfloor \frac{\lfloor x+y \rfloor + \lfloor 2y-x \rfloor + 2}{3} \right\rfloor. \tag{3}$$

One can verify that

$$\begin{aligned} k_1 &= k - q_1 c + q_2 d, \\ k_2 &= q_2 d - q_1 d - q_2 c \end{aligned} \tag{4}$$

satisfy $k \equiv k_1 + k_2\lambda \pmod{n}$ and $|k_1|, |k_2| < \frac{2\sqrt{n}}{3}$, which is known to be optimal [19].

For each k, the computation of x and y requires divisions. We propose a simplification using a one-time pre-computation (once per curve) that eliminates these divisions. Let

$$s = \lceil \log_2 n \rceil, \qquad t = \lceil \log_2(2(|c| + |d|)) \rceil,$$

and pre-compute three constants

$$\eta_1 = \left\lfloor \frac{(c-2d)2^{s+t}}{n} \right\rfloor, \quad \eta_2 = \left\lfloor \frac{(2c-d)2^{s+t}}{n} \right\rfloor, \quad \eta_3 = \left\lfloor \frac{(-c-d)2^{s+t}}{n} \right\rfloor.$$

Then for a given $0 \le k < n$, we compute $f_j = \left\lfloor \frac{k\eta_j}{2^{s+t}} \right\rfloor$ for $j = 1, 2, 3$, and obtain q_1, q_2 as

$$q_1 = \left\lfloor \frac{f_1 + f_2 + 2}{3} \right\rfloor, \qquad q_2 = \left\lfloor \frac{f_1 + f_3 + 2}{3} \right\rfloor. \tag{5}$$

Substituting (q_1, q_2) into (4) gives the decomposition (2). Finally, a decomposition in terms of $\tau = 1 - \lambda$ is obtained by

$$k \equiv k_1 + k_2\lambda \pmod{n} = (k_1 + k_2) + (-k_2)\tau \pmod{n}. \tag{6}$$

Remarks.

1. Only three multiplications $k\eta_1$, $k\eta_2$, $k\eta_3$ are needed; division by 2^{s+t} is trivial.

2. The integers q_1, q_2 are close to x, y; for instance,

$$3x - \frac{1}{2^{t-1}} \leq f_1 + f_2 + 2 < 3x + 2.$$

Jacobian projective coordinates. For $(X, Y, Z) \in \mathbb{F}_p^3$, Jacobian projective coordinates $(X : Y : Z)$ denote the equivalence class

$$(X : Y : Z) = \{(\lambda^2 X, \lambda^3 Y, \lambda Z) : \lambda \in \mathbb{F}_p^*\}.$$

If $Z \neq 0$, the corresponding affine coordinates are $(\frac{X}{Z^2}, \frac{Y}{Z^3})$. The curve equation becomes

$$E_b : Y^2 = X^3 + bZ^6, \tag{7}$$

with the point at infinity represented as $(1 : 1 : 0)$.

For two non-trivial points $P = (X_P : Y_P : Z_P)$ and $Q = (X_Q : Y_Q : Z_Q)$ with $P \neq \pm Q$, the sum $R = P + Q = (X_R : Y_R : Z_R)$ is given by

$$\begin{aligned}
X_R &= (Y_Q Z_P^3 - Y_P Z_Q^3)^2 - (X_Q Z_P^2 - X_P Z_Q^2)^2 (X_P Z_Q^2 + X_Q Z_P^2), \\
Y_R &= (Y_Q Z_P^3 - Y_P Z_Q^3)\big(X_P Z_Q^2 (X_Q Z_P^2 - X_P Z_Q^2)^2 - X_R\big) \\
&\quad - Y_P Z_Q^3 (X_Q Z_P^2 - X_P Z_Q^2)^3, \\
Z_R &= (X_Q Z_P^2 - X_P Z_Q^2) Z_P Z_Q.
\end{aligned}$$

For doubling $(P = Q)$,

$$\begin{aligned}
X_R &= 9X_P^4 - 8X_P Y_P^2, \\
Y_R &= 3X_P^2 (4X_P Y_P^2 - X_R) - 8Y_P^4, \\
Z_R &= 2Y_P Z_P.
\end{aligned}$$

Based on the improved formulas of Longa and Miri [24], the costs in Jacobian coordinates for curves $E_b/\mathbb{F}_p$ are

$$1\mathbf{ADD} = 5\mathbf{S} + 11\mathbf{M}, \qquad 1\mathbf{DBL} = 5\mathbf{S} + 2\mathbf{M}.$$

The procedures are summarized in Table 3. Throughout this paper we adopt the conservative conversion $1\mathbf{S} = 1\mathbf{M}$ for sidechannel resistant implementations; thus $1\mathbf{ADD} = 16\mathbf{M}$ and $1\mathbf{DBL} = 7\mathbf{M}$. In settings where sidechannel resistance is not a concern, one may instead use $1\mathbf{S} = 0.8\mathbf{M}$ or less [4, 5].

Table 3: Point addition and doubling for $y^2 = x^3 + b$ in Jacobian coordinates

Addition $R = P + Q$ $(P \neq \pm Q)$	**Double** $R = 2P$
$A = X_P Z_Q^2 - X_Q Z_P^2$;	$A = 3X_P^2$;
$B = Y_P Z_Q^3 - Y_Q Z_P^3$;	$B = 2Y_P Z_P$;
$C = 2X_Q Z_P^2 A^2$;	$C = B^2$;
$D = 4Y_Q Z_P^3 A^3$;	$D = CX_P$ (or $(X_P + C)^2 - X_P^2 - C^2)/2$);
$X_R = 4(B^2 - A^3 - C)$;	$X_R = A^2 - 2D$;
$Y_R = 2(B(2C - X_R) - D)$;	$Y_R = (D - X_R)A - C^2/2$;
$Z_R = ((A + Z_Q)^2 - A^2 - Z_Q^2)Z_P$;	$Z_R = BZ_P$;

3 Efficient Formulas

The curve $E_b/\mathbb{F}_p$ with prime $p \equiv 1 \pmod 3$ has complex multiplication by $\mathbb{Z}[\omega]$, and multiplication by ω costs only $1\mathbf{M}$. It is possible that multiplication by other elements of $\mathbb{Z}[\omega]$ with small norm is efficient as well. It is thus natural to consider evaluations by $\tau = 1 - \omega$, $\rho = 2 - \omega$, and $\sigma = 3 - \omega$, which are prime factors of the rational primes 3, 7, and 13, respectively.

The computation of τP is indeed simple and its formula in affine coordinates was given in [1] (see also [2]). In the first part of this section, we show that in Jacobian projective coordinates, τP can be computed with $3\mathbf{S} + 3\mathbf{M}$. This will lead to a much more efficient formula for tripling. In the second part of this section, we develop efficient formulas for the endomorphisms $2 - \omega$ and $3 - \omega$, as well as for multiplication by 7.

We now consider multiplications by τ and 3. The affine formula in the next proposition is from [1].

Proposition 3.1. *Let $P = (x, y) \in E_b(\mathbb{F}_p)$.*

1. *The affine coordinates of τP are*

$$\begin{cases} x' = \dfrac{x^3 + 4b}{(1-\beta)^2 x^2} \\ y' = y\dfrac{x^3 - 8b}{(1-\beta)^3 x^3}, \end{cases}$$

   where $\beta \in \mathbb{F}_p$ satisfies $\beta^3 = 1, \beta \neq 1$.

2. *In Jacobian projective coordinates, τP can be computed using $3\mathbf{S} + 3\mathbf{M}$.*

3. *In Jacobian projective coordinates, $3P$ can be computed using $4\mathbf{S} + 6\mathbf{M}$.*

Proof. 1. For $P = (x, y)$, we have $\omega P = (\beta x, y)$. To compute $\tau P = P - \omega P = (x', y')$, we first see that the slope is $\ell = \frac{2y}{(1-\beta)x}$.

Now notice that $(1-\beta)^2 = -3\beta$; we have

$$\begin{aligned} x' &= \ell^2 - (1+\beta)x = -\frac{4y^2}{3\beta x^2} - (1+\beta)x \\ &= -\frac{\big(4 + 3\beta(1+\beta)\big)x^3 + 4b}{3\beta x^2} = \frac{x^3 + 4b}{(1-\beta)^2 x^2}. \end{aligned}$$

$$\begin{aligned} y' &= \ell(x - x') - y = \frac{2y}{(1-\beta)x}\left(\frac{3\beta x^3}{3\beta x^2} + \frac{x^3 + 4b}{3\beta x^2}\right) - y \\ &= y\,\frac{2(3\beta+1)x^3 - 3\beta(1-\beta)x^3 + 8b}{3\beta(1-\beta)x^3} = y\,\frac{x^3 - 8b}{(1-\beta)^3 x^3}. \end{aligned}$$

2. To derive a formula in Jacobian coordinates, write $P = \big(\frac{X}{Z^2} : \frac{Y}{Z^3} : 1\big)$. Then

$$\begin{aligned} X' &= \frac{\big(\frac{X}{Z^2}\big)^3 + 4b}{(1-\beta)^2\big(\frac{X}{Z^2}\big)^2} = \frac{X^3 + 4bZ^6}{((1-\beta)XZ)^2}, \\ Y' &= \frac{Y}{Z^3}\,\frac{\big(\frac{X}{Z^2}\big)^3 - 8b}{(1-\beta)^3\big(\frac{X}{Z^2}\big)^3} = \frac{Y(X^3 - 8bZ^6)}{((1-\beta)XZ)^3}. \end{aligned}$$

Thus Jacobian coordinates (X_τ, Y_τ, Z_τ) for τP can be taken as

$$\begin{aligned} X_\tau &= X^3 + 4bZ^6, \\ Y_\tau &= Y(X^3 - 8bZ^6), \\ Z_\tau &= (1-\beta)XZ. \end{aligned}$$

Using $bZ^6 = Y^2 - X^3$ and $(1-\beta)^2 = -3\beta$, this reduces to

$$\begin{aligned} X_\tau &= 4Y^2 - 3X^3 \\ Y_\tau &= Y(9X^3 - 8Y^2) = Y(-2X_\tau + 3X^3) \\ Z_\tau &= (1-\beta)XZ = Z\frac{(1-\beta+X)^2 - X^2 + 3\beta}{2} \end{aligned} \tag{8}$$

Computing X_τ costs $2\mathbf{S} + 1\mathbf{M}$, Y_τ costs $1\mathbf{M}$, and Z_τ costs $1\mathbf{S} + 1\mathbf{M}$ with the square computation of $(1 - \beta + X)$.

3. By the fact that $3 = (1-\omega)(1-\omega^2)$, we have $3P = (1-\omega^2)(\tau P)$.

   Let $(X_3 : Y_3 : Z_3)$ be the Jacobian coordinates of $3P$ computed from $(1-\omega^2)(\tau P)$. Similar to the above,

$$\begin{aligned} X_3 &= 4Y_\tau^2 - 3X_\tau^3, \\ Y_3 &= Y_\tau(9X_\tau^3 - 8Y_\tau^2) = Y_\tau(-2X_3 + 3X_\tau^3), \\ Z_3 &= (1-\beta^2)X_\tau Z_\tau. \end{aligned}$$

   Assuming we have already computed X_τ, Y_τ, the computation of X_3, Y_3 costs $2\mathbf{S} + 2\mathbf{M}$ (as in the previous step). For Z_3, note that

$$Z_3 = (1-\beta^2)X_\tau Z_\tau = (1-\beta^2)X_\tau(1-\beta)XZ = 3X_\tau(XZ),$$

   so without explicitly computing Z_τ, we can obtain Z_3 with $2\mathbf{M}$. Adding the cost for X_τ, Y_τ (which is $2\mathbf{S} + 2\mathbf{M}$ from step 2), the total for $3P$ becomes $4\mathbf{S} + 6\mathbf{M}$. □

Remarks.

1. In Jacobian projective coordinates for a general Weierstrass form, a previously known cost for calculating $3P$ for general curves over prime fields is given in [24, 5]. When restricted to $E_b/\mathbb{F}_p$, the best prior cost is $10\mathbf{S} + 5\mathbf{M}$. Our specific formula achieves a lower cost of $4\mathbf{S} + 6\mathbf{M}$.

2. It is interesting to note that doubling uses similar building blocks as τP, since the doubling formulas for E_b in Jacobian coordinates are

$$\begin{aligned} X_2 &= X(9X^3 - 8Y^2), \\ Y_2 &= 9X^3(4Y^2 - 3X^3) - 8Y^4, \\ Z_2 &= 2YZ. \end{aligned} \tag{9}$$

   We also note that the optimal cost for doubling is $5\mathbf{S} + 2\mathbf{M}$ [5].

3. Several facts about τ suggest the possibility of developing a window τ-NAF method, which has been very successful for the family of Koblitz curves over binary fields, for scalar multiplication on $E_b/\mathbb{F}_p$. These facts include: (1) $N(\tau) > 1$; (2) τP is faster than point doubling; and (3) any integer $k < n$ admits an efficient reduction to a compact form $k_1 + k_2\tau$.

   In the last step of computing the Jacobian coordinates of τP, we need

$$Z_\tau = (1-\beta)XZ.$$

If ωP is also available, we can obtain βX as an extra input and compute $(1-\beta)X = X - \beta X$, saving one multiplication. This reduces the cost of τP to $2\mathbf{S} + 3\mathbf{M}$; we call this an *augmented-τ* operation.

We shall propose a window τ-NAF method for scalar multiplication on $E_b/\mathbb{F}_p$ in the next section. Due to some nice properties of the Eisenstein integers, we are able to construct a very efficient pre-computation. The window τ-NAF can be further optimized by using tripling and the augmented-τ operation.

Now we consider the evaluation of the endomorphism $\rho = 2 - \omega$.

Proposition 3.2. *Let $P = (x, y) \in E_b(\mathbb{F}_p)$.*

1. *The affine coordinates of ρP are*

$$\begin{cases} x' = x\left(\dfrac{-27x^6 - 12(1+3\beta)(2+\beta)x^3y^2 + 16(1+3\beta)y^4}{\beta^4[4y^2 - 3(1-\beta)x^3]^2}\right) \\ y' = y\left(\dfrac{-81(1+3\beta)(2+\beta)x^9 + 2^2 3^3 7\beta x^6 y^2 - 144(1+3\beta)x^3y^4 + 64y^6}{\beta^6[4y^2 - 3(1-\beta)x^3]^3}\right); \end{cases}$$

2. *In Jacobian projective coordinates, ρP can be computed using $6\mathbf{S} + 7\mathbf{M}$;*

3. *In Jacobian projective coordinates, $7P$ can be computed using $12\mathbf{S} + 14\mathbf{M}$.*

Proof. 1. We use the formula from Proposition 3.1 in the calculation

$$\rho P = P + \tau P = (x, y) + \left(\frac{x^3 + 4b}{(1-\beta)^2 x^2}, y\frac{x^3 - 8b}{(1-\beta)^3 x^3}\right).$$

With the slope $\kappa = \dfrac{y\left(1 - \frac{x^3-8b}{(1-\beta)^3x^3}\right)}{x - \frac{x^3+4b}{(1-\beta)^2x^2}} = \dfrac{2y}{(1-\beta)x}\dfrac{(2+3\beta)x^3 - 4b}{(1+3\beta)x^3 + 4b}$, we get

$$\begin{aligned} x' &= \kappa^2 - x - \frac{x^3 + 4b}{(1-\beta)^2x^2} = \kappa^2 - \frac{(1-3\beta)x^3 + 4b}{((1-\beta)x)^2} \\ &= \frac{4y^2[(2+3\beta)x^3 - 4b]^2 - ((1-3\beta)x^3 + 4b)[(1+3\beta)x^3 + 4b]^2}{((1-\beta)x[(1+3\beta)x^3 + 4b])^2} \\ &= x\left(\frac{-27x^6 - 12(1+3\beta)(2+\beta)x^3y^2 + 16(1+3\beta)y^4}{\beta[4y^2 - 3(1-\beta)x^3]^2}\right). \end{aligned}$$

The y-coordinate of ρP can be directly computed as [1]

$$\begin{aligned} y' &= \kappa(x-\overline{x})-y = \frac{2y}{(1-\beta)x}\frac{(2+3\beta)x^3-4b}{(1+3\beta)x^3+4b}(x-\overline{x})-y \\ &= y\left(\frac{-81(1+3\beta)(2+\beta)x^9+756\beta x^6y^2-144(1+3\beta)x^3y^4+64y^6}{[4y^2-3(1-\beta)x^3]^3}\right). \end{aligned}$$

2. To derive a formula in Jacobian coordinates, write $P=(\frac{X}{Z^2}:\frac{Y}{Z^3}:1)$. Then

$$x' = \frac{X\big(-27X^6-12(1+3\beta)(2+\beta)X^3Y^2+16(1+3\beta)Y^4\big)}{\beta^4Z^2\big(4Y^2-3(1-\beta)X^3\big)^2},$$

$$y' = \frac{Y\big((81-324\beta)X^9+756\beta X^6Y^2-144(1+3\beta)X^3Y^4+64Y^6\big)}{\beta^6Z^3\big(4Y^2-3(1-\beta)X^3\big)^3}.$$

So a Jacobian coordinates (X_ρ, Y_ρ, Z_ρ) for ρP becomes

$$\begin{aligned} X_\rho &= X\big(-27X^6-12(1+3\beta)(2+\beta)X^3Y^2+16(1+3\beta)Y^4\big) \\ Y_\rho &= Y\big((81-324\beta)X^9+756\beta X^6Y^2-144(1+3\beta)X^3Y^4+64Y^6\big) \\ Z_\rho &= \beta^2Z\big(4Y^2-3(1-\beta)X^3\big) \end{aligned} \tag{10}$$

We first compute $A=3X^3, B=4Y^2, A^2, B^2, C=\beta A=\frac{(A+\beta)^2-A^2-\beta^2}{2}, D=\beta B=\frac{(B+\beta)^2-B^2-\beta^2}{2}$ with $\mathbf{M}+6\mathbf{S}$. Then

$$\begin{aligned} X_\rho &= X\big(-3A^2-(B+3D)(2A-B+C)\big) \\ Y_\rho &= Y\big(3A^2(A-4C+7D)+B^2(B-3A-9C)\big) \\ Z_\rho &= Z\big(2A-B+C-D\big). \end{aligned}$$

With this arrangement, computing X_ρ, Y_ρ and Z_ρ use $2\mathbf{M}, 3\mathbf{M}$, and $\mathbf{M}$ respectively. So the entire operation of ρP requires $6\mathbf{S}+7\mathbf{M}$.

3. By the fact that $7=(2-\omega)(2-\omega^2)$, we have $7P=(2-\omega^2)(\rho P)$.

Let $(X_7:Y_7:Z_7)$ be the Jacobian projective coordinates of $7P$ computed from $(2-\omega^2)(\rho P)$. In this case, we just need to use β^2 in the position of β in (10). Thus

$$\begin{aligned} X_7 &= X_\rho\big(-27X_\rho^6-12(1+3\beta^2)(2+\beta)X_\rho^3Y_\rho^2+16(1+3\beta^2)Y_\rho^4\big) \\ Y_7 &= Y_\rho\big((81-324\beta^2)X_\rho^9+756\beta^2X_\rho^6Y_\rho^2-144(1+3\beta^2)X_\rho^3Y_\rho^4+64Y_\rho^6\big) \\ Z_7 &= \beta Z_\rho\big(4Y_\rho^2-3(1-\beta^2)X_\rho^3\big) \end{aligned}$$

[1] Another way of computing the y-coordinate of ρP is to compute $\frac{dx'}{dx}$. In general, let $(k+\ell\omega)P=(R(x), yS(x))$, then $S(x)=\frac{1}{k+\ell\beta}\frac{dR(x)}{dx}$.

Given ρP, the computation of $7P$ also takes $6\mathbf{S} + 7\mathbf{M}$. This means that the computation of $7P$ from P requires $12\mathbf{S} + 14\mathbf{M}$.

□

Remarks.

1. We first remark that ρP is less expensive than a point addition as the latter costs $5\mathbf{S} + 11\mathbf{M}$. We shall use it in the pre-computation later in the window τ-NAF. It is also remarked that the $7P$ formula in proposition 3.2 is also efficient. As an independent calculation of $7P$ could be $7P = 2^3P - P$ and it costs $3(5\mathbf{S} + 2\mathbf{M}) + (5\mathbf{S} + 11\mathbf{M}) = 20\mathbf{S} + 17\mathbf{M}$.

2. We note that in finding ρP, one needs to calculate X^3, Y^2 and Y^4 as intermediate result. These terms also appear when using (9) to calculate $2P$. If both ρP and $2P$ are required, they can be proceeded together to save $1\mathbf{M} + 3\mathbf{S}$.

3. We also remark that in forming the affine coordinates of ρP (similar to τP), we manage to make the denominators of x and y in the way so that the conversion to Jacobian projective coordinates becomes more natural.

For the endomorphism $\sigma = 3 - \omega$, we just state the calculation result of σP here:

Proposition 3.3. *Let $P = (x, y) \in E_b(\mathbb{F}_p)$. The affine coordinates of $\sigma P = (3-\omega)P$ are*

$$\begin{aligned}
x' &= \frac{\beta x^{16} + (-336 - 199\beta)bx^{13} + (-480 - 1400\beta)b^2x^{10} + (-720 - 2672\beta)b^3x^7}{y^2[(4+3\beta)x^6 + 4(5-6\beta)bx^3 + 16b^2]^2} \\
&\quad + \frac{(-1344 - 1216\beta)b^4x^4 + (-768 + 256\beta)b^5x}{y^2[(4+3\beta)x^6 + 4(5-6\beta)bx^3 + 16b^2]^2} \\
y' &= \frac{-x^{24} + (298 + 756\beta)bx^{21} + (-18649 - 1944\beta)b^2x^{18} + (-65396 - 26892\beta)b^3x^{15}}{y^3[(4+3\beta)x^6 + 4(5-6\beta)bx^3 + 16b^2]^3} \\
&\quad + \frac{(-36016 - 38016\beta)b^4x^{12} + (69568 - 62208\beta)b^5x^9 + (76544 - 103680\beta)b^6x^6}{y^3[(4+3\beta)x^6 + 4(5-6\beta)bx^3 + 16b^2]^3} \\
&\quad + \frac{(13312 - 55296\beta)b^7x^3 - 4096b^8}{y^3[(4+3\beta)x^6 + 4(5-6\beta)bx^3 + 16b^2]^3}
\end{aligned}$$

At the end of this section, we include a table to summarize the costs of some efficient endomorphisms we have discussed:

Table 4: Costs of Endomorphisms

Endomorphism	Our Formula	Current Known Result	Remark
$\tau = 1 - \omega$	$3\mathbf{S} + 3\mathbf{M}$	–	Better than a point doubling
3	$4\mathbf{S} + 6\mathbf{M}$	$10\mathbf{S} + 5\mathbf{M}$	
$\rho = 2 - \omega$	$6\mathbf{S} + 7\mathbf{M}$	–	Better than a point addition
7	$12\mathbf{S} + 14\mathbf{M}$	$20\mathbf{S} + 17\mathbf{M}$	

4 Window τ-NAF Method

We have identified an efficient endomorphism $\tau = 1 - \omega$ for E_b with norm $N(\tau) = 3 > 1$. It is a desirable candidate for creating an efficient window τ-NAF method, as it satisfies the basic requirements of [16].

In [22], Koblitz considered the following supersingular elliptic curve over finite fields of characteristic 3:

$$E_{3,a} : y^2 = x^3 - x - (-1)^a \quad \text{over } \mathbb{F}_{3^m}, \quad a \in \{0, 1\}.$$

The Frobenius map in this case is $\tau_3(x, y) = (x^3, y^3)$ (with $\tau_3(\mathcal{O}) = \mathcal{O}$), and its characteristic polynomial is $T^2 - 3(-1)^a T + 3$. Thus $\mathbb{Z}[\omega]$ is also the endomorphism ring of $E_{3,a}$. Although τ_3 and τ are different endomorphisms (they act on different elliptic curves), they are the same as complex numbers. Koblitz proved the existence and uniqueness of a non-adjacent form for τ_3-adic expansions of integers in $\mathbb{Z}[\tau_3] = \mathbb{Z}[\omega]$, and used this form to perform fast scalar multiplication on $E_{3,a}(\mathbb{F}_{3^m})$.

A window τ_3-NAF method for $E_{3,a}$ was later proposed by Blake, Murty and Xu [6], where a reduction procedure and a termination proof for the window τ_3-NAF algorithm were given. In this section we further develop some of the ideas from [6]; however, owing to the different nature of $E_b/\mathbb{F}_p$ and $E_{3,a}/\mathbb{F}_{3^m}$ we adopt the reduction described in Section 2. For a window size of 4, we carefully design an efficient pre-computation for the τ-NAF on $E_b(\mathbb{F}_p)$. (We note that no concrete pre-computation is discussed in [6].) In this paper, we employ our efficient tripling ($3P$) and augmented-τ operations to achieve further gains.

Given a positive integer w, the width-w window τ-NAF method seeks an expansion (called a *window τ-NAF*) of an element $k_1 + k_2\tau \in \mathbb{Z}[\omega]$ of the form

$$k_1 + k_2\tau = \sum_{j=0}^{N} c_j \tau^j \tag{11}$$

such that each nonzero coefficient c_j belongs to a suitable set C (the *coefficient set*) and every block $\{c_j, c_{j+1}, \ldots, c_{j+w-1}\}$ contains at most one non-zero entry. To com-

pute $(k_1 + k_2\tau)P$, one first performs a pre-computation: evaluate cP for every $c \in C$, and then evaluate $\sum_{j=0}^{N} \tau^j(c_j P)$.

4.1 Criterion of Divisibility by τ^w

To derive the expansion (11), a criterion for divisibility of elements in $\mathbb{Z}[\tau]$ by τ^w is useful. The next lemma is from [6, 8], where a proof was outlined. Since it is crucial for our discussion, we provide a more detailed proof here.

Lemma 4.1. *Let w be a positive integer and $\delta_w = \lceil \frac{w}{2} \rceil - \lfloor \frac{w}{2} \rfloor$ (so $\delta_w = 0$ if w is even, and $\delta_w = 1$ if w is odd). Then*

1. $\tau^w = (-3\omega)^{\lfloor \frac{w}{2} \rfloor} \tau^{\delta_w}$.
2. *For $x + y\tau \in \mathbb{Z}[\tau]$,*

$$\tau^w | x + y\tau \iff 3^{\lceil \frac{w}{2} \rceil} | x \text{ and } 3^{\lfloor \frac{w}{2} \rfloor} | y.$$

Proof. 1. Since $\tau^2 = -3\omega$, we have

$$\tau^w = (\tau^2)^{\lfloor \frac{w}{2} \rfloor} \tau^{\delta_w} = (-3\omega)^{\lfloor \frac{w}{2} \rfloor} \tau^{\delta_w}.$$

2. Assume $\tau^w \mid x + y\tau$. Because $(-\omega)^{\lfloor \frac{w}{2} \rfloor}$ is a unit, we have

$$3^{\lfloor \frac{w}{2} \rfloor} \tau^{\delta_w} \mid x + y\tau.$$

Hence there exist $\alpha, \beta \in \mathbb{Z}$ such that

$$x + y\tau = 3^{\lfloor \frac{w}{2} \rfloor} (\alpha \tau^{\delta_w} + \beta \tau^{1+\delta_w}).$$

We consider the parity of w.

Case w even ($\delta_w = 0$): Then $x + y\tau = 3^{\frac{w}{2}}(\alpha + \beta\tau)$, so

$$x = 3^{\lfloor \frac{w}{2} \rfloor} \alpha = 3^{\lceil \frac{w}{2} \rceil} \alpha, \qquad y = 3^{\lfloor \frac{w}{2} \rfloor} \beta.$$

Case w odd ($\delta_w = 1$): Then since $\tau^2 = -3 + 3\tau$

$$x = -3^{\lfloor \frac{w}{2} \rfloor + 1} \beta = -3^{\lceil \frac{w}{2} \rceil} \beta, \quad y = 3^{\lfloor \frac{w}{2} \rfloor} (\alpha + 3\beta).$$

In both cases we obtain

$$3^{\lceil \frac{w}{2} \rceil} \mid x \text{ and } 3^{\lfloor \frac{w}{2} \rfloor} \mid y.$$

Conversely, suppose $x = 3^{\lceil \frac{w}{2} \rceil} f, y = 3^{\lfloor \frac{w}{2} \rfloor} g$ for some integers f, g. Then

$$\begin{aligned} x + y\tau &= 3^{\lfloor \frac{w}{2} \rfloor}(3^{\delta_w} f + g\tau) \\ &= 3^{\lfloor \frac{w}{2} \rfloor}(-\omega)^{\lfloor \frac{w}{2} \rfloor} \tau^{\delta_w} \frac{3^{\delta_w} f + g\tau}{\tau^{\delta_w}} (-\omega)^{-\lfloor \frac{w}{2} \rfloor} \\ &= \tau^w \left(\frac{3^{\delta_w} f + g\tau}{\tau^{\delta_w}} (-\omega)^{-\lfloor \frac{w}{2} \rfloor} \right), \end{aligned}$$

where the last factor belongs to $\mathbb{Z}[\tau]$ due to the fact that $\tau \mid 3$. Hence $\tau^w \mid x + y\tau$. □

The following corollary is useful for designing an efficient set of coefficients.

Corollary 4.1. *Let $w > 1$ and let $a + b\tau \not\equiv 0 \pmod{\tau}$. Then for any unit $u \in U \setminus \{1\}$ (where U is the unit group of $\mathbb{Z}[\omega]$),*

$$a + b\tau \not\equiv u(a + b\tau) \pmod{\tau^w}.$$

Proof. Since $a + b\tau \not\equiv 0 \pmod{\tau}$, we have $3 \nmid a$ (because $\tau \mid a + b\tau$ iff $3 \mid a$). For each $u \neq 1$, we compute $x + y\tau = a + b\tau - u(a + b\tau)$:

u	$x + y\tau$
-1	$2a + 2b\tau$
ω	$-3b + (3b + a)\tau$
$-\omega$	$(2a + 3b) - (a + b)\tau$
ω^2	$3(a + b) - a\tau$
$-\omega^2$	$-(a + 3b) + (a + 2b)\tau$

In each case, because $3 \nmid a$, one can verify that either $3^{\lceil \frac{w}{2} \rceil} \nmid x$ or $3^{\lfloor \frac{w}{2} \rfloor} \nmid y$. By Lemma 4.1, this means $\tau^w \nmid (x + y\tau)$, so $a + b\tau \not\equiv u(a + b\tau) \pmod{\tau^w}$. □

4.2 Pre-computation

We need to decide a set for the coefficients of the representation (11).

Consider the set of all elements of $\mathbb{Z}[\tau]$ that are not divisible by τ. By Lemma 4.1, a set of representatives of the congruence classes of such elements modulo τ^w is

$$R = \left\{ x + y\tau \ : \ 0 \le x \le 3^{\lceil \frac{w}{2} \rceil} - 1, \ 0 \le y \le 3^{\lfloor \frac{w}{2} \rfloor} - 1, \ 3 \nmid x \right\}.$$

For each $x + y\tau \in R$, we are interested in elements $g + h\tau \in \mathbb{Z}[\tau]$ that satisfy

$$g + h\tau \equiv x + y\tau \pmod{\tau^w}, \quad \text{and} \quad N(g + h\tau) < 3^w.$$

This leads us to define

$$C_{x,y} = \left\{ g + h\tau \in \mathbb{Z}[\tau] \ : \ g \equiv x \pmod{3^{\lceil \frac{w}{2} \rceil}},\ h \equiv y \pmod{3^{\lfloor \frac{w}{2} \rfloor}},\ N(g + h\tau) < 3^w \right\}.$$

Since $\mathbb{Z}[\tau]$ is a Euclidean ring, $C_{x,y}$ is non-empty. In fact, $C_{x,y}$ usually contains more than one element. This flexibility allows us to select an element from $C_{x,y}$ that leads to a better pre-computation for the coefficients in the expansion (11).

For each $x + y\tau \in R$, we choose one element $\tilde{x} + \tilde{y}\tau$ from each $C_{x,y}$. Then a set of non-zero coefficients for the expression (11) can be formed as follows:

$$C = \left\{ \tilde{x} + \tilde{y}\tau \ : \ x + y\tau \in R \right\}. \tag{12}$$

We can use Algorithm 3.1 of [6] to produce the expansion (11). The correctness of this algorithm is examined in [6] (Theorem 3.1): because $\mathbb{Z}[\omega]$ is Euclidean and imaginary quadratic, the width-w window τ-NAF method terminates. We include this algorithm below.

Algorithm 1 Width w window τ-NAF method

Require: an element $\eta = a + b\tau$ of $\mathbb{Z}[\tau]$
Ensure: S, the array of coefficients of window τ-NAF of η

function GEN-NAF(a, b)
    $S \leftarrow <>$
    while $a \neq 0$ or $b \neq 0$ **do**
        if $3 \nmid a$ **then**
            $x \leftarrow a \pmod{3^{\lceil \frac{w}{2} \rceil}}$
            $y \leftarrow b \pmod{3^{\lfloor \frac{w}{2} \rfloor}}$
            $a \leftarrow a - \tilde{x}$
            $b \leftarrow b - \tilde{y}$
            prepend $\tilde{x} + \tilde{y}\tau$ to S
        else
            prepend 0 to S
        end if
        $t \leftarrow a$
        $a \leftarrow a + b$
        $b \leftarrow \frac{-t}{3}$
    end while
    return S
end function

4.2.1 A pre-computation for $w = 4$.

We now describe an explicit efficient pre-computation for $w = 4$.

Let $w = 4$. In this case, we need to find easily computable coefficients $\tilde{x} + \tilde{y}\tau$ from each $C_{x,y}$ for $x = 1, 2, 4, 5, 7, 8$ and $y \in \{0, 1, 2, \cdots, 8\}$. It is noted that only $x = 1, 2, 4$ should be considered, as

$$-(x + y\tau) \equiv (9 - x) + (9 - y)\tau \pmod{\tau^4}.$$

The part for $x = 5, 7, 8$ is obtained by negation. This means that we only need to determine 27 coefficients and half of the pre-computations are saved. As can be seen later, there are actually 9 pre-computations that need to be processed, the rest 18 can be obtained easily. More specifically, with each $x+y\tau$ ($x = 1, 2, 4$ and $0 \leq y < 9$) serving as an index for the corresponding coefficient $\tilde{x} + \tilde{y}\tau$, we first fix 9 coefficients

$$S = \{1, 2, 4, 1 + \tau, 2 + 2\tau, 1 + 2\tau, 2 + 4\tau, 2 + \tau, 1 - 2\tau\},$$

these coefficients correspond to the representatives $1, 2, 4, 1 + \tau, 2 + 2\tau, 1 + 2\tau, 2 + 4\tau, 2 + \tau, 1 + 7\tau$. The set of (non-zero) coefficients for (11) is given by

$$C = S \cup (-S) \cup \omega S \cup (-\omega S) \cup \omega^2 S \cup (-\omega^2 S).$$

By corollary 4.1, the set C represents all classes (modulo τ^4) of elements that are not divisible by τ. Since the unit group of $\mathbb{Z}[\omega]$ is $U = \{1, -1, \omega, -\omega, \omega^2, -\omega^2\}$, we have

$$C = UC,$$

that is, C is U-invariant.

In table 5, the first, third and fifth row are all indexes $x + y\tau$'s for $x = 1, 2, 4$ and $0 \leq y < 9$. The second row lists the set S, with their corresponding indexes $x + y\tau$ in the same column of the first row. An element $\tilde{x} + \tilde{y}\tau$ in the forth row is obtained by multiplying ω to the corresponding (column) element in the second row if $\tilde{x} + \tilde{y}\tau \in C_{x,y}$ with $1 \leq x \leq 4$, otherwise, $\tilde{x} + \tilde{y}\tau$ is obtained by multiplying $-\omega$ to the corresponding (column) element in the second row. The corresponding representative $x + y\tau$ is put in the third row as its index. We form the sixth row and fifth row in a similar way by multiplying ω^2 or $-\omega^2$ to the corresponding (column) element in the second row.

Precisely, coefficients $\tilde{x}+\tilde{y}\tau$ in the fourth row in table 5 are obtained by multiplying ω or $-\omega$ to the ones in the second row in the following manner:

$1 - \tau = \omega \cdot 1$,	$2(1 - \tau) = \omega \cdot 2$,	$4(1 - \tau) = \omega \cdot 4$
$4 - 3\tau = \omega \cdot (1 + \tau)$,	$-8 + 6\tau = -\omega \cdot (2 + 2\tau)$,	$-7 + 5\tau = -\omega \cdot (1 + 2\tau)$
$-14 + 10\tau = -\omega \cdot (2 + 4\tau)$,	$-5 + 4\tau = -\omega \cdot (2 + \tau)$	$-5 + 3\tau = \omega \cdot (1 - 2\tau)$.

Table 5: Pre-Computation for $w = 4$

$x + y\tau$	1	2	4	$1+\tau$	$2+2\tau$	$1+2\tau$	$2+4\tau$	$2+\tau$	$1+7\tau$
$\tilde{x} + \tilde{y}\tau$	1	2	4	$1+\tau$	$2+2\tau$	$1+2\tau$	$2+4\tau$	$2+\tau$	$1-2\tau$
$x + y\tau$	$1+8\tau$	$2+7\tau$	$4+5\tau$	$4+6\tau$	$1+6\tau$	$2+5\tau$	$4+\tau$	$4+4\tau$	$4+3\tau$
$\tilde{x} + \tilde{y}\tau$	$1-\tau$	$2(1-\tau)$	$4(1-\tau)$	$4-3\tau$	$-8+6\tau$	$-7+5\tau$	$-14+10\tau$	$-5+4\tau$	$-5+3\tau$
$x + y\tau$	$2+8\tau$	$4+7\tau$	$1+4\tau$	$4+2\tau$	$1+5\tau$	$1+3\tau$	$2+6\tau$	$2+3\tau$	$4+8\tau$
$\tilde{x} + \tilde{y}\tau$	$2-\tau$	$4-2\tau$	$-8+4\tau$	$-5+2\tau$	$10-4\tau$	$-8+3\tau$	$-16+6\tau$	$-7+3\tau$	$4-\tau$

Similarly, $\tilde{x} + \tilde{y}\tau$'s in the sixth row in table 5 are obtained :

$$\begin{array}{lll} 2-\tau = -\omega^2 \cdot 1 & 4-2\tau = -\omega^2 \cdot 2, & -8+4\tau = \omega^2 \cdot 4 \\ -5+2\tau = \omega^2 \cdot (1+\tau), & 10-4\tau = -\omega^2 \cdot (2+2\tau), & -8+3\tau = \omega^2 \cdot (1+2\tau) \\ -16+6\tau = \omega^2 \cdot (2+4\tau), & -7+3\tau = \omega^2 \cdot (2+\tau), & 4-\tau = \omega^2 \cdot (1-2\tau). \end{array}$$

We denote C^+ the set of coefficients appeared in table 5 (the second, forth and sixth rows). Then

$$C = C^+ \cup (-C^+).$$

Given a point P, the pre-computation is to compute $Q_{x+y\tau} = (\tilde{x} + \tilde{y}\tau)P$ for $x + y\tau \in C^+$ (i.e. $x = 1, 2, 4$ and $0 \le y < 9$).

From the construction, we see that if a point $Q = (\tilde{x} + \tilde{y}\tau)P$ for the second row is computed, then the corresponding pre-computation in the fourth row and sixth row can be simply obtained as ωQ (or $-\omega Q$) and $\omega^2 Q$ (or $-\omega^2 Q$) , respectively. Note that computing ωQ is easy: if $Q = (r, s)$, then $\omega Q = (\beta r, s)$; computing $\omega^2 Q$ can even be neglected in the sense that

$$\omega^2 Q = (\beta^2 r, s) = (-\beta r - r, s),$$

since the multiplication βr has been computed in ωQ.

Concretely, we perform $Q_{x+y\tau} = (\tilde{x} + \tilde{y}\tau)P$ for the second row one by one in the following order:

$$\begin{array}{ll} Q_2 = 2P, & Q_4 = 4P = 2Q_2, \\ Q_{1+\tau} = (1+\tau)P = \rho P, & Q_{2+2\tau} = (2+2\tau)P = 2Q_{1+\tau}, \\ Q_{2+\tau} = (2+\tau)P = Q_{1+\tau} + P, & Q_{1+2\tau} = (1+2\tau)P = Q_{2+2\tau} - P, \\ Q_{1+7\tau} = (1-2\tau)P = Q_2 - Q_{1+2\tau} & Q_{2+4\tau} = (2+4\tau)P = 2Q_{1+2\tau}. \end{array}$$

In this process, we compute $2P$ and ρP together to use $16\mathbf{M}$ (as they share the computations of X^3, Y^2, Y^4). The rest requires $3\mathbf{DBL} + 3\mathbf{ADD}$ (e.g., $Q_4 = 4P = 2Q_2, Q_{2+2\tau} = 2Q_{1+\tau}, Q_{2+4\tau} = 2Q_{1+2\tau}$ for doubles; $Q_{2+\tau} = (2+\tau)P = Q_{1+\tau} +$

$P, Q_{1+2\tau} = Q_{2+2\tau} - P, Q_{1+7\tau} = Q_2 - Q_{1+2\tau}$ for additions). The computation for the fourth row needs 9 field multiplications. So the pre-computation cost is

$$16\mathbf{M} + 9\mathbf{M} + 3 \cdot 7\mathbf{M} + 3 \cdot 16\mathbf{M} = 94\mathbf{M}.$$

4.3 Width-4 Window τ-NAF Method for Scalar Multiplication

In coordination with the pre-computation designed above, we choose window size $w = 4$ for scalar multiplication. This is an appropriate window size because it also helps to maximize the benefit brought by the more efficient $3P$ formula. We have determined the set C^+ of coefficients in section 4.2.1 for pre-computation. The pre-computation with respect to a point $P \in E_b(\mathbb{F}_p)$ is to compute cP for every $c \in C^+$. The scalar multiplication with negative part of C^+ is trivial.

Now we describe our scalar multiplication algorithm for curves $E_b/\mathbb{F}_p$ over prime fields. Given a scalar $k < n$ and a point P of order n, we first use (6) to get reduction $k = k_1' + k_2'\lambda \pmod{n}$. This implies that $kP = (k_1' + k_2'\omega)P$. Let $k_1 = k_1' + k_2', k_2 = -k_2'$, then

$$kP = (k_1 + k_2\tau)P.$$

Next we use Algorithm 1 to compute

$$k_1 + k_2\tau = \sum_{i=0}^{\ell-1} \varepsilon_i c_i \tau^i \tag{13}$$

where $c_i \in C^+ \cup \{0\}, \varepsilon_i \in \{-1, 1\}$. The window τNAF algorithm is

Algorithm 2 Window τ-NAF Scalar Multiplication Method for $E_b/\mathbb{F}_p$

Require: a positive integer $k < n$ and a point $P \in E_b(\mathbb{F}_p)$ of order n
Ensure: the scalar multiplication kP

```
1: function SCAL-MUL(k, P)
2:     Pre-Computation: cP for each c ∈ C^+.
3:     Get expression (13) for k
4:     R ← O
5:     for i from ℓ − 1 downto 0 do
6:         R ← τR
7:         if c_i ≠ 0 then
8:             if ε_i = 1 then
9:                 R ← R + c_i P
```

```
            else
                R ← R − c_i P
            end if
        end if
    end for
    return R
end function
```

4.3.1 An optimization.

The expression (13) can be written as

$$k_1 + k_2\tau = \varepsilon_{\ell_0} c_{\ell_0} \tau^{\ell_0} + \varepsilon_{\ell_1} c_{\ell_1} \tau^{\ell_1} + \cdots + \varepsilon_{\ell_{t-1}} c_{\ell_{t-1}} \tau^{\ell_{t-1}}$$

where the nonzero coefficient $c_{\ell_j} \in C^+$, $\varepsilon_{\ell_j} \in \{1, -1\}$, and $\ell_j - \ell_{j-1} \geq 4$.

By lemma 4.1, $\tau^\ell = (-3\omega)^{\lfloor \frac{\ell}{2} \rfloor} \tau^{\delta_\ell}$ where $\delta_\ell = \lceil \frac{\ell}{2} \rceil - \lfloor \frac{\ell}{2} \rfloor$, the above expression of $k_1 + k_2\tau$ can be rewritten as

$$\begin{aligned} k_1 + k_2\tau &= \varepsilon_{\ell_0} c_{\ell_0} (-\omega)^{\lfloor \frac{\ell_0}{2} \rfloor} \tau^{\delta_{\ell_0}} 3^{\lfloor \frac{\ell_0}{2} \rfloor} + \varepsilon_{\ell_1} c_{\ell_1} (-\omega)^{\lfloor \frac{\ell_1}{2} \rfloor} \tau^{\delta_{\ell_1}} 3^{\lfloor \frac{\ell_1}{2} \rfloor} + \\ &\quad + \cdots + \varepsilon_{\ell_{t-1}} c_{\ell_{t-1}} (-\omega)^{\lfloor \frac{\ell_{t-1}}{2} \rfloor} \tau^{\delta_{\ell_{t-1}}} 3^{\lfloor \frac{\ell_{t-1}}{2} \rfloor} \\ &= \varepsilon'_{\ell_0} c'_{\ell_0} \tau^{\delta_{\ell_0}} 3^{\lfloor \frac{\ell_0}{2} \rfloor} + \varepsilon'_{\ell_1} c'_{\ell_1} \tau^{\delta_{\ell_1}} 3^{\lfloor \frac{\ell_1}{2} \rfloor} + \\ &\quad + \cdots + \varepsilon'_{\ell_{t-1}} c'_{\ell_{t-1}} \tau^{\delta_{\ell_{t-1}}} 3^{\lfloor \frac{\ell_{t-1}}{2} \rfloor} \end{aligned} \tag{14}$$

Here $\varepsilon'_{\ell_j} c'_{\ell_j} = \varepsilon_{\ell_j} c_{\ell_j} (-\omega)^{\lfloor \frac{\ell_j}{2} \rfloor} \in C$ since C is U-invariant, and the element $\varepsilon'_{\ell_j} \in \{1, -1\}$ is chosen so that $c'_{\ell_j} \in C^+$. The new expression (14) makes the use of the efficient tripling possible (recall that $3R$ can be done by using only $10\mathbf{M}$ for any point R). In the calculation of kP, the term $\varepsilon'_{\ell_j} c'_{\ell_j} \tau^{\delta_{\ell_j}} P$ is processed as $\tau^{\delta_{\ell_j}} (\varepsilon'_{\ell_j} c'_{\ell_j} P) = \tau^{\delta_{\ell_j}} (Q_{x+y\tau})$ where $Q_{x+y\tau} = \varepsilon'_{\ell_j} c'_{\ell_j} P$ has already been included in pre-computation. By the U-invariant property, if $Q_{x+y\tau}$ is pre-computed, so is $\omega Q_{x+y\tau}$. Thus if $\delta_{\ell_j} = 1$, one can use the augmented-τ to compute $\tau^{\delta_{\ell_j}} (Q_{x+y\tau})$ using $5\mathbf{M}$, which is half of that for $3P$.

The optimized version of our window τNAF algorithm is

Algorithm 3 Optimized Window τ-NAF Scalar Multiplication Method for $E_b/\mathbb{F}_p$

Require: a positive integer $k < n$ and a point $P \in E_b(\mathbb{F}_p)$ of order n
Ensure: the scalar multiplication kP
1: **function** OPT-SCAL-MUL(k, P)

Pre-Computation: cP for each $c \in C^+$.
Get expression (14) for k
$R \leftarrow \mathcal{O}$
for i from $\lfloor \frac{\ell_{t-1}}{2} \rfloor$ downto 0 **do**
    $R \leftarrow 3R$
    if $i = \lfloor \frac{\ell_j}{2} \rfloor$ for some j **then**
        $Q \leftarrow c'_{\ell_j} P$
        if $\delta_{\ell_j} = 1$ **then**
            $Q \leftarrow \tau Q$ (*Use augmented-τ since ωQ is available*)
        end if
        if $\varepsilon'_{\ell_j} = 1$ **then**
            $R \leftarrow R + Q$
        else
            $R \leftarrow R - Q$
        end if
    end if
end for
return R
end function

4.3.2 Estimation of the cost for width-4 window τ-NAF.

In the process of calculating kP for $k < n$, one reduces k to $k_1 + k_2\tau$ such that $k \equiv k_1 + k_2\tau \pmod{n}$ with $|k_1| \lessapprox \frac{4}{3}\sqrt{n}, |k_2| \lessapprox \frac{2}{3}\sqrt{n}$, as in (6).

Note that $N(\tau) = 3$, the window τ-NAF for $k_1 + k_2\tau$ is expected to be of length $\log_3 n$. In the expansion (11) for $k_1 + k_2\tau$, after each nonzero coefficient, there will be w consecutive zeros, beyond that, asymptotic expectation of zeros followed is $\frac{1}{3} + \frac{1}{3^2} + \cdots = \frac{1}{2}$. So the average number of nonzero terms is

$$\frac{\log_3 n}{w + \frac{1}{2}}.$$

For $w = 4$, the average density of nonzero digits is approximately $\frac{1}{4.5}$ and an expansion of length approximately $\log_3 n$ contains on average $\frac{\log_3 n}{4.5}$ nonzero term. Therefore, given the required precomputation, algorithm 2 requires $2\frac{\log_3 n}{4.5}$ **tripling**, $(\log_3 n - 4\frac{\log_3 n}{4.5})$ **augmented-τ**, and $\frac{\log_3 n}{4.5}$**ADD**.

Recall that under the assumption that $1\mathbf{S} = 1\mathbf{M}$, **augmented-$\tau$** $= 5\mathbf{M}$, **tripling** $= 10\mathbf{M}$, and **ADD** $= 16\mathbf{M}$, so the overall cost for the computation of kP in terms of

the number of field multiplications becomes

$$\left(\frac{2}{4.5}\cdot 10 \quad +(1-\tfrac{4}{4.5})\cdot 5+\tfrac{1}{4.5}\cdot 16\right)\log_3 n\mathbf{M}+94\mathbf{M}$$
$$<(5.40\log_2 n+94)\mathbf{M}. \tag{15}$$

Currently, the best method for scalar multiplication for curves $E_b/\mathbb{F}_p$ is the GLV method. According to our experiments, the version of interleaving with 4-NAF performs the best for SECP256K1. For $E_b/\mathbb{F}_p$, the interleaving with w-NAF is to compute k_1P+k_2Q where $Q=\lambda P$ using w-NAF to k_1 and k_2. The pre-computation can be done essentially just for P: pre-compute jP for $j=3,5,\cdots,2^{w-1}-1$. The Q part of pre-computation is obtained simply by $jQ=\lambda(jP)$ by using 4 field multiplications for $j=1,3,5,7$. The rest of the cost of interleaving with 4-NAF is $\frac{\log_2 n}{2}\mathbf{DBL}+\frac{\log_2 n}{5}\mathbf{ADD}$. Converting to field multiplications, the cost becomes

$$\left(\frac{\log_2 n}{2}+1\right)\cdot 7\mathbf{M}+\left(\frac{\log_2 n}{5}+3\right)\cdot 16\mathbf{M}+4\mathbf{M}=(6.7\log_2 n+59)\mathbf{M}. \tag{16}$$

The concrete numbers of field multiplications used for the cases of $\log_2 n=256,384$ and 512 are summarized in Table 1. Thus, comparing (15) with (16), we see that the width-4 window τ-NAF method achieves more than $16.7\%, 17.6\%$ and 18% of improvements over the GLV method, when $\log_2 n=256,384$ and 512 respectively.

4.3.3 Regular window τ-NAF

In [20], Joye and Tunstall described regular recoding algorithms for integers to achieve side-channel resistant exponentiation. This idea was extended to τ-NAF recoding for Koblitz curves by Oliveira, Aranha, López, and Rodríguez-Henríquez [27] (see also [28]).

The regular window τ-NAF representation of a scalar k is the following sparse τ-expansion of its reduction $k_1+k_2\tau$:

$$k_1+k_2\tau=\sum_{i=0}^{\ell-1}u_i(\tau^w)^i, \tag{17}$$

where each coefficient $u_i\neq 0$. For Koblitz curves E_a, this is achieved in [27] by taking the coefficient set given in [32] for τ-NAF with window size $w+1$. The essential property is that the 2^w coefficients lie in distinct equivalence classes modulo τ^{w+1}, and they are split into 2^{w-1} pairs where the two coefficients in each pair are congruent modulo τ^w.

Yu and Xu [36] use a different approach by choosing two coefficient sets for τ-NAF with window size w, so that the Euclidean property of $\mathbb{Q}(\sqrt{-7})$[2] can be used to guarantee the correctness of the coefficient selection process [6]. The union of these two sets shares the same essential property as the construction in [27].

Here we follow the idea of [36] to construct two coefficient sets for the case $w = 4$ that satisfy the required condition to obtain a regular window τ-NAF representation for curves $E_b/\mathbb{F}_p$. More specifically, for each pair (x, y) with $0 \le x \le 8$, $0 \le y \le 8$, and $3 \nmid x$, we select two elements

$$U_{x+y\tau} = \tilde{x} + \tilde{y}\tau, \qquad V_{x+y\tau} = \tilde{\tilde{x}} + \tilde{\tilde{y}}\tau$$

from the set

$$C_{x,y} = \big\{ g + h\tau \in \mathbb{Z}[\tau] \ : \ g \equiv x \pmod{3^2},\ h \equiv y \pmod{3^2},\ N(g + h\tau) < 3^4 \big\}.$$

To form a regular expression (17) for $w = 4$, we require

$$\tau^5 \nmid (V_{x+y\tau} - U_{x+y\tau}). \tag{18}$$

Note that $\tau^4 \mid (V_{x+y\tau} - U_{x+y\tau})$, so the elements in

$$\{V_{x+y\tau},\, U_{x+y\tau} \ : \ 0 \le x \le 8,\ 0 \le y \le 8,\ 3 \nmid x\}$$

are all distinct modulo τ^5. Moreover, there are 54 pairs $(V_{x+y\tau}, U_{x+y\tau})$ that are congruent modulo τ^4.

By Lemma 4.1, condition (18) reduces to

$$\tilde{x} \not\equiv \tilde{\tilde{x}} \pmod{3^3}.$$

For one set we use $C = \{\tilde{x} + \tilde{y}\tau \ : \ 0 \le x \le 8,\ 0 \le y \le 8,\ 3 \nmid x\}$ from Section 4.2.1. The second set is

$$C' = C'^{+} \cup (-C'^{+}),$$

where

$$\begin{aligned} C'^{+} = &\{-8,\, -7,\, -5,\, -8+\tau,\, -7+2\tau,\, -8+2\tau,\, -7+4\tau,\, -7+\tau,\, -8+7\tau\} \\ &\cup \{-8+8\tau,\, -7+7\tau,\, -5+5\tau,\, -5+6\tau,\, 1-3\tau,\, 2-4\tau,\, -5+\tau,\, 4-5\tau\} \\ &\cup \{-16+8\tau,\, -14+7\tau,\, 10-5\tau,\, 13-7\tau,\, -8+5\tau,\, 10-6\tau,\, 2-3\tau\} \\ &\cup \{11-6\tau,\, -5-\tau\}. \end{aligned}$$

[2]Recall that the Frobenius endomorphism for the Koblitz curve $E_a/\mathbb{F}_{2^m}$ is identified with the complex number $\frac{(-1)^{1-a}+\sqrt{-7}}{2}$.

The set C' is U-invariant as well.

The pre-computation order is arranged similarly to Table 5. The coefficients $\tilde{\tilde{x}}+\tilde{\tilde{y}}\tau$ and their indices $x+y\tau$ are given in Table 6.

Table 6: Another Pre-Computation for $w = 4$

$x+y\tau$	1	2	4	$1+\tau$	$2+2\tau$	$1+2\tau$	$2+4\tau$	$2+\tau$	$1+7\tau$
$\tilde{\tilde{x}}+\tilde{\tilde{y}}\tau$	-8	-7	-5	$-8+\tau$	$-7+2\tau$	$-8+2\tau$	$-7+4\tau$	$-7+\tau$	$-8+7\tau$
$x+y\tau$	$1+8\tau$	$2+7\tau$	$4+5\tau$	$4+6\tau$	$1+6\tau$	$2+5\tau$	$4+\tau$	$4+4\tau$	$4+3\tau$
$\tilde{\tilde{x}}+\tilde{\tilde{y}}\tau$	$-8+8\tau$	$-7+7\tau$	$-5+5\tau$	$-5+6\tau$	$1-3\tau$	$2-4\tau$	$-5+\tau$	$4-5\tau$	$13-6\tau$
$x+y\tau$	$2+8\tau$	$4+7\tau$	$1+4\tau$	$4+2\tau$	$1+5\tau$	$1+3\tau$	$2+6\tau$	$2+3\tau$	$4+8\tau$
$\tilde{\tilde{x}}+\tilde{\tilde{y}}\tau$	$-16+8\tau$	$-14+7\tau$	$10-5\tau$	$13-7\tau$	$-8+5\tau$	$10-6\tau$	$2-3\tau$	$11-6\tau$	$-5-\tau$

Similarly, we only need to compute all $Q'_{x+y\tau} = (\tilde{\tilde{x}} + \tilde{\tilde{y}}\tau)P$ for the second row of Table 6 as

$$
\begin{aligned}
&Q'_1 = -8P = -2Q_4, && Q'_2 = -7P = Q'_1 + P,\\
&Q'_4 = -5P = -Q_4 - P, && Q'_{1+\tau} = (-8+\tau)P = -Q_{1+8\tau} + Q'_2,\\
&Q'_{2+2\tau} = (-7+2\tau)P = -Q_{2+7\tau} + Q'_4, && Q'_{1+2\tau} = (-8+2\tau)P = -2Q_{4+8\tau},\\
&Q'_{2+4\tau} = (-7+4\tau)P = Q_{4+4\tau} - Q_2, && Q'_{2+\tau} = (-7+\tau)P = Q'_{1+\tau} + P,\\
&Q'_{1+7\tau} = (-8+7\tau)P = Q'_{2+7\tau} - P.
\end{aligned}
$$

This requires $2\mathbf{DBL} + 7\mathbf{ADD}$. Pre-computation corresponding to the fourth and sixth rows (of Table 6) are obtained by multiplying the x-coordinate of the above $Q'_{x+y\tau}$ by β (or $-\beta$) and β^2 (or $-\beta^2$). Since $\beta^2 = -\beta - 1$, this requires an additional $9\mathbf{M}$.

To obtain a regular τ-expansion, we rewrite lines 7-9 of Algorithm 1 to get

Algorithm 4 Width-w Regular Window τ-NAF Algorithm

Require: an element $\eta = a + b\tau$ of $\mathbb{Z}[\tau]$

Ensure: S, the array of coefficients of regular window τ-NAF of η

1: **function** GEN-NAF(a, b)
2: $S \leftarrow <>$
3: **while** $a \neq 0$ or $b \neq 0$ **do**
4: **if** $3 \nmid a$ **then**
5: $x \leftarrow a \pmod{3^{\lceil \frac{w}{2} \rceil}}$
6: $y \leftarrow b \pmod{3^{\lfloor \frac{w}{2} \rfloor}}$

```
            if ((a = x̃) and (b = ỹ)) or ((a = x̃̃) and (b = ỹ̃)) then
                prepend a + bτ to S
                break
            end if
            if 3^3 | (a − x̃) then
                a ← a − x̃̃
                b ← b − ỹ̃
                prepend x̃̃ + ỹ̃τ to S
            else
                a ← a − x̃
                b ← b − ỹ
                prepend x̃ + ỹτ to S
            end if
        else
            prepend 0 to S
        end if
        t ← a
        a ← a + b
        b ← −t/3
    end while
    return S
end function
```

Fixing $w = 4$ in (17), we may also assume that $3 \nmid k_1$ (otherwise consider the expansion of $(k_1 - 1) + k_2\tau$), so the regular form for our case is

$$k_1 + k_2\tau = \varepsilon_0 c_0 + \varepsilon_1 c_1 \tau^4 + \varepsilon_2 c_2 \tau^8 + \cdots + \varepsilon_r c_r \tau^{4r}$$

with $c_i \in C \cup C'$. Applying $\tau^{4j} = \omega^{2j} 3^{2j}$, we obtain the more convenient form

$$k_1 + k_2\tau = \varepsilon_0 c_0 + \varepsilon_1 c_1' 3^2 + \varepsilon_2 c_2' 3^4 + \cdots + \varepsilon_r c_r' 3^{2r}$$

where $c_j' = \omega^{2j} c_j \in C \cup C'$, since $C \cup C'$ is U-invariant.

Finally, we estimate the cost of the regular version. The pre-computation for the second coefficient set is $2\mathbf{DBL}+7\mathbf{ADD}+9\mathbf{M} = 135\mathbf{M}$. Thus the scalar multiplication using our new method requires

$$\left(\frac{2}{4} \cdot 10 + \frac{1}{4} \cdot 16\right) \log_3 n \, \mathbf{M} + 94\mathbf{M} + 135\mathbf{M} < (5.68 \log_2 n + 229)\, \mathbf{M}.$$

The constant-time GLV method with window size 5 requires

$$\left(\frac{\log_2 n}{2} + 1\right) \cdot 7\mathbf{M} + \left(\frac{\log_2 n}{4} + 7\right) \cdot 16\mathbf{M} + 8\mathbf{M} = (7.5 \log_2 n + 127)\, \mathbf{M}.$$

Table 2 lists the precise numbers of **M** used for group orders of 256, 384, and 512 bits, respectively. It indicates that the constant-time scalar multiplication using our regular window τ-NAF improves over the regular GLV method by 17.7%, 19.8% and 20.9%, respectively. For larger values of $\log_2 n$, the improvement exceeds 24%.

We also developed an independent C implementation of $E_b/\mathbb{F}_p$, including the standard curves secp256k1 and BLS12-381 (source code is available in the supplemental materials). Experiments were performed on an Ubuntu 24.04 system with an Intel Core Ultra 9 185H 2.5 GHz processor. The program was compiled using GCC 14.2.0 and GMP 6.3.0. For the regular version of scalar multiplication, the execution time is consistent with the above estimates. Table 7 presents a performance comparison, where the values in the second and third columns denote the average runtime in milliseconds (ms).

Table 7: Comparison of Regular Scalar Multiplication

Curve	Our method	GLV method	Improvement
secp256k1	0.148ms	0.180ms	17.8%
BLS12-381	0.208ms	0.261ms	20.3%

5 Conclusion

Finding an efficiently computable endomorphism τ suitable for constructing a window τ-NAF for elliptic curves over prime fields has been a long-sought interesting problem. In this paper, we identify such an endomorphism by considering nonzero nonunit elements of $\mathbb{Z}[\omega]$ of minimal norm for the curves $E_b : y^2 = x^3 + b$ over $\mathbb{F}_p$ for a prime $p \equiv 1 \pmod 3$. We show that $\tau = 1 - \omega$ satisfies the basic requirements for being an appropriate candidate. The cost of evaluating τP is comparable to that of the Frobenius map on curves over extension fields of small characteristic. This also leads to an efficient formula for tripling. In addition, we obtain several other useful endomorphisms, including $\rho = 2 - \omega$, $\sigma = 3 - \omega$, and 7. In Jacobian projective coordinates, the costs of evaluating τ and ρ are lower than those of point doubling and point addition, respectively. Our formulas for $3P$ and $7P$ also significantly improve upon existing results.

To design a window τ-NAF method for the family of curves $E_b/\mathbb{F}_p$, we develop some mathematical tools and create a very efficient pre-computation. To the best of our knowledge, this is the first window τ-NAF scalar multiplication method developed for curves over prime fields in which τ is a complex algebraic integer rather than a rational integer.

Our pre-computation is based on a carefully designed coefficient set whose U-invariant property is repeatedly exploited to optimize the performance of the window τ-NAF method for $E_b/\mathbb{F}_p$. In terms of the number of field multiplications, for group orders of 256, 384, and 512 bits, our algorithm achieves performance improvements exceeding 16.7%, 17.6%, and 18%, respectively, compared with the GLV method.

To resist simple side-channel attacks, we further describe a regular window τ-NAF method. This version of scalar multiplication achieves improvements of 17.7%, 19.8%, and 20.9% for group orders of 256, 384, and 512 bits, respectively.

References

[1] Antony, S. N. F. M. A. and Kamarulhaili, H.: Improvement of scalar multiplication on elliptic curve with j-invariant 0, *International Journal of Cryptology Research*, 10(2020), 22-36.

[2] Antony, S. N. F. M. A., Yion, C. H. K., Kamarulhaili, H., Ariffin, M. R. K., and Yunos, F. Fast Endomorphisms in Integer Sub-Decomposition Method on Secp192k1, *Malaysian Journal of Mathematical Sciences*, 18 (2024), 501-514. Barreto, P.S.L.M., Lynn, B., and Scott, M.: Constructing elliptic curves with prescribed embedding

[3] degrees. In Security in Communication Networks 2002, Lecture Notes in Computer Science, vol. 2576, 263-273.

[4] Bernstein, D. J., Chuengsatiansup, C., and Lange, T.: Double-base scalar multiplication revisited. https://eprint.iacr.org/2017/037.

[5] Bernstein, D. J. and Lange, T.: The explicit-formulas database, 2025, https://www.hyperelliptic.org/EFD/.

[6] Blake, I. F., Murty, K. V., Xu, G.: A note on window τ-NAF algorithm, *Information Processing Letters*, 95(2005), no. 5, 496-502.

[7] Blake I. F., Murty K. V., Xu G.: Efficient algorithms for Koblitz curves over fields of characteristic three, *Journal of Discrete Algorithms*, 3(2005), 113-124.

[8] Blake, I. F., Murty, K. V., Xu, G.: Nonadjacent radix-τ expansions of integers in Euclidean imaginary quadratic number fields, *Canadian Journal of Mathematics*, 60(2008), 1267-1282.

[9] Bos, J. W., Costello, C., Hisil, H., and Lauter, K.: High-performance scalar multiplication using 8-dimensional GLV/GLS decomposition. In Cryptographic

Hardware and Embedded Systems CHES 2013, pp. 331348. Springer, Berlin, Heidelberg.

[10] Bowe, S.: BLS12-381: New zk-SNARK elliptic curve construction, 2017, https://electriccoin.co/blog/new-snark-curve.

[11] Brown, E., Myers, B.T., Solinas, J.A.: Elliptic curves with compact parameters. Tech. Report, Centre for Applied Cryptographic Research (2001). http://www.cacr.math.uwaterloo.ca/techreports/2001/ corr2001-68.ps.

[12] Ciet M., Sica F., and Quisquater J.-J.: Analysis of the Gallant-Lambert-Vanstone method based on efficient endomorphisms: Elliptic and hyperelliptic curves. In Nyberg, K. and Heys, H. M. (eds), Selected Areas in Cryptography: SAC 2002. Lecture Notes in Computer Science, vol. 2595, 21-36.

[13] Cohen H.: A course in computational algebraic number theory. Springer, 2000.

[14] Doche, C., Icart, T., Kohel, D.R.: Efficient scalar multiplication by isogeny decompositions. Yung, M., Dodis, Y., Kiayias, A., Malkin, T. (eds.) Public Key Cryptography-PKC 2006. Lecture Notes in Computer Science, vol. 3958, 191-206.

[15] Galbraith S.D., Lin X.B., Scott M.: Endomorphisms for faster elliptic curve cryptography on a Large class of curves. Journal of Cryptology, 24(2011), 446-469.

[16] Gallant, R.P., Lambert, R.J., Vanstone, S.A.: Faster point multiplication on elliptic curves with efficient endomorphisms. In: Kilian, J., (ed.) Advances in Cryptology - CRYPTO 2001. Lecture Notes in Computer Science, vol. 2139, pp. 190-200. Springer, (2001).

[17] Hankerson, D., Menezes, A., Vanstone, S.: Guide to elliptic curve cryptography. New York, NY, USA: Springer-Verlag, 2004.

[18] Hisil, H., Carter, G. P., and Dawson, E. P.: New Formulae for Efficient Elliptic Curve Arithmetic. In Srinathan, K., Rangan, C. P., Yung, M. (Eds.): Indocrypt 2007, LNCS 4859, pp. 138-151.

[19] Hu, Z., Longa, P., Xu, M.: Implementing the 4-dimensional GLV method on GLS elliptic curves with j-invariant 0. Des., Codes Cryptography, 63(2012) , 331-343.

[20] Joye, M., Tunstall, M.: Exponent recoding and regular exponentiation algorithms. In: Preneel, B. (ed.) Advances in Cryptology-AFRICACRYPT 2009. Lecture Notes in Computer Science, vol. 5580, 334-349. Springer, Heidelberg (2009)

[21] Koblitz, N.: CM-curves with good cryptographic properties. In: Feigenbaum, J. (eds) Advances in Cryptology-CRYPTO 1991. Lecture Notes in Computer Science, vol. 576, 279-287. Springer, Berlin, Heidelberg. https://doi.org/10.1007/3-540-46766-1_22

[22] Koblitz, N.: An elliptic curves implementation of the finite field digital signature algorithm. In Krawczyk, H. (eds) Advances in Cryptology-CRYPTO'98. Lecture Notes in Computer Science, vol. 1462, 327-337. Springer-Verlag Berlin Heidelberg, (1998).

[23] Koblitz, N.: Algebraic aspects of cryptography, Springer, 1999.

[24] Longa P. and Miri A.: Fast and flexible elliptic curve point arithmetic over prime fields, *IEEE Transactions on Computers*, 57(2008) , 289-302.

[25] Longa, P., Sica, F.: Four-dimensional gallant-lambert-vanstone scalar multiplication. Advances in Cryptology-ASIACRYPT 2012. Beijing, China: Springer, 718-739, 2012.

[26] Longa, P., Sica, F.: Four-dimensional gallant-lambert-vanstone scalar multiplication. Journal of Cryptology, 27(2014), 248-283.

[27] Oliveira, T., Aranha, D.F., López, J., Rodríguez-Henríquez, F.: Fast point multiplication algorithms for binary elliptic curves with and without precomputation. SAC 2014, LNCS, vol. 8781, 324-344. Springer, Heidelberg, 2014.

[28] Oliveira, T., López, J., Cervantes-Vázquez, D., Rodríguez-Henríquez, F.: Software implementation of Koblitz curves over quadratic fields. Journal of Cryptology, 32(2019), 867-894.

[29] Rajwade, A. R.: On rational primes p congruent to 1 (mod 3 or 5), Proc. Cambridge Philos. Sot. 66 (1969), 61-70.

[30] Smith, B.: Easy scalar decompositions for efficient scalar multiplication on elliptic curves and genus 2 Jacobians, https://eprint.iacr.org/2013/672.pdf.

[31] Smith, B.: Families of fast elliptic curves from Q-curves. In Sako, K. and Sarkar, P. (eds) Advances in Cryptology - ASIACRYPT 2013, Part I. Lecture Notes in Computer Science, vol. 8269, 61-78. https://doi.org/10.1007/978-3-642-42033-7_4.

[32] Solinas, J.: Efficient arithmetic on Koblitz curves, Des., Codes Cryptography, 19(2000) , 195-249.

[33] Takagi, T., Yen, S.-M. and Wu, B.-C.: Radix-r non-adjacent form, Information Security Conference – ISC 2004. Lecture Notes in Computer Science, vol. 3225, pp. 99-110. Springer-Verlag, Berlin, 2004.

[34] Trost, W. and Xu, G.: On the optimal pre-computation of window τNAF for Koblitz curves. IEEE Transactions on Computers. Vol. 65, No. 9. 2016, 2918-2924.

[35] Wu, H. and Xu, G.: On Koblitz curves over prime fields. Journal of Cryptologic Research, 2024, Vol. 11, Issue (5), 1152-1159. DOI: 10.13868/j.cnki.jcr.000735

[36] Yu, W. and Xu, G.: Pre-computation scheme of window τNAF for Koblitz curves revisited. In Canteaut, A. and Standaert, F.-X. (eds) Advances in Cryptology–EUROCRYPT 2021, Part II. Springer Cham, Lecture Notes in Computer Science, vol. 12697, 187-218. DOI: 10.1007/978-3-030-77886-6_7

[37] SP 800-186 recommendations for discrete logarithm-based cryptography: elliptic curve domain parameters, *https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-186.pdf*,2023.

[38] SEC 2: recommended elliptic curve domain parameters, *https://www.secg.org/sec2-v2.pdf*, 2010.